Práctica 4: Programación entera con Pyomo.

1. Introducción

En esta práctica se mostrará como resolver problemas de programación entera utilizando Pyomo. Se trabajará con ejemplos ya que es más sencilla la explicación

2. Ejemplo de programación entera

Para resolver un problema de programación entera (mixta) simplemente definiremos las variables necesarias dentro del dominio correspondiente:

- **Variables binarias**: se utilizará como dominio el conjunto `Binary` del entorno `pyomo.environ`
- **Variables enteras**: se utilizará como dominio el conjunto `Integers` del entorno `pyomo.environ` (o también los conjuntos `NonNegativeIntegers`, `NegativeIntegers`, `NonPositiveIntegers`, ...)

A continuación se resolverá el siguiente problema de programación entera:

$$\begin{array}{rrcl} \textbf{minimizar} & -5x_1 - 8x_2 & & \\ \textbf{sujeto a} & x_1 + x_2 & \leq & 6 \\ & 5x_1 + 9x_2 & \leq & 45 \\ & \boldsymbol{x} & \geq & 0 \\ & \boldsymbol{x} & \in & \mathbb{Z}^2. \end{array}$$

El archivo a utilizar será `P_entera.py` (se puede descargar de la página web de la asignatura) y su contenido será:

```python
# Modelo concreto para un ejemplo de programación entera
#   Ejemplo

#   minimizar z=-5x1 - 8x2
#   s. a.
#              x1 +  x2 <=  6
#             5x1 + 9x2 <= 45
#              x1,x2 >= 0
#              x1,x2 in Z


import pyomo.environ as pyo

# Crear el modelo
model = pyo.ConcreteModel()

# Definir las variables y su dominio PARA EL PROBLEMA RELAJADO
# Si se quiere resolver el problema entero hay que comentar las dos líneas siguientes
# y descomentar las de después.
model.x1 = pyo.Var(domain=pyo.NonNegativeReals)
model.x2 = pyo.Var(domain=pyo.NonNegativeReals)

# # Definir las variables y su dominio
# model.x2 = pyo.Var(domain=pyo.NonNegativeIntegers)
# model.x1 = pyo.Var(domain=pyo.NonNegativeIntegers)

# Establecer la función objetivo
model.Objetivo = pyo.Objective(expr = -5*model.x1 - 8*model.x2, sense=pyo.minimize)

# Establecer las restricciones
model.Rest1 = pyo.Constraint(expr = model.x1 + model.x2 <= 6)
```

```
model.Rest2 = pyo.Constraint(expr = 5*model.x1 + 9*model.x2 <= 45)


# Para mostrar las soluciones ejecutar 'pyomo solve' en linea de comandos con las opciones:
# antes de ello debeéis de comentar todas las líneas que vienen después de la siguiente:
# pyomo solve --solver=glpk --show-results P_Entera.py


# Para que se muestren directamente al ejecutar desde python
# Habría que escribir en el prompt: python P_Entera.py
# Descomentar las siguientes líneas, sólo el primer '#' de cada línea.

solver = pyo.SolverFactory('glpk')

results = solver.solve(model)  # Añadir ,options={'nomip':''} para que resuelva el problema
    relajado (solo glpk)

# Imprime la información sobre la ejecución del solver
print(results)

# Salida formateada:
print("\nSolución:")
print(f"\tCoste = {model.Objetivo()}")

print("\nVariables en el óptimo:")
for v in model.component_objects(pyo.Var, active=True):
    print(f"\t{v} = {v()}")


print("\nRestricciones en el óptimo:")
for c in model.component_objects(pyo.Constraint, active=True):
    print(f"\t{c} = {c()}")

# Muestra el modelo sin resolver
#model.pprint()

# Muestra el modelo resuelto
#model.display()
```

Para obtener la solución se ejecutaría el siguiente código en el *prompt* de Anaconda:

```
python P_entera.py
```

El resultado se mostrará por pantalla.

3. Modelos abstractos: El problema de la mochila

En esta sección introduciremos un modelo abstracto para el problema de la mochila y resolveremos algún ejemplo con datos concretos.

En un problema de la mochila tenemos n objetos que queremos introducir en una mochila de capacidad limitada P (peso o volumen). De cada objeto j sabemos su peso/volumen p_j y la utilidad u_j (dinero) que nos proporciona si el objeto se introduce en la mochila. Si las variables de decisión son tales que $x_j = 1$ si el objeto j se mete en la mochila y $x_j = 0$ en otro caso (*Variables binarias*), entonces la formulación del problema genérico sería:

$$\begin{aligned} \textbf{maximizar} \quad & \sum_{j=1}^{n} u_j x_j \\ \textbf{sujeto a} \quad & \sum_{j=1}^{n} p_j x_j \leq P \\ & \boldsymbol{x} \in \{0,1\}^n. \end{aligned}$$

El archivo a utilizar será `P_entera.py` (se puede descargar de la página web de la asignatura) y su contenido será:

```
# Modelo abstracto para el problema de la mochila

import pyomo.environ as pyo

# Crear el modelo
model = pyo.AbstractModel()

# Definir los Parámetros
model.J = pyo.Set() # Conjunto de objetos

model.u = pyo.Param(model.J)  # Utilidad de cada objeto
model.p = pyo.Param(model.J, within=pyo.NonNegativeReals)  # Peso/volumen de cada objeto
model.P = pyo.Param(within=pyo.NonNegativeReals) # Capacidad (peso o volumen máximo) de la
    mochila


# Definir las variables y su dominio
model.x = pyo.Var(model.J, domain=pyo.Binary)

# Establecer la función objetivo en dos pasos:
# 1. se define la expresión de la función objetivo
def obj_expression(m):
    return pyo.summation(m.u, m.x)
# 2. Se establece la función objetivo
model.Objetivo = pyo.Objective(rule=obj_expression,sense=pyo.maximize)

# Establecer la restricción de capacidad:
# 1. se define la expresión del bloque de restricciones
def capacidad_constraint_rule(m):
    return pyo.summation(m.p, m.x) <= m.P

# 2. Se crea la restricción de capacidad
model.Capacidad = pyo.Constraint(rule=capacidad_constraint_rule)


# Para mostrar las soluciones ejecutar 'pyomo solve' en linea de comandos con la opción --
    summary:
# pyomo solve --solver=glpk --show-results Mochila.py Mochila_1.dat

#############################################################################################
# Para que se muestren directamente al ejecutar desde python (con  p ej. IDLE)
solver = pyo.SolverFactory('glpk')
# Se crea una instancia: modelo abstracto + datos
instancia = model.create_instance('Mochila_1.dat')
#Se resuleve la instancia
results = solver.solve(instancia) # Añadir ,options={'nomip':''} para que resuelva el problema
     relajado (solo glpk)
# Imprime la información sobre la ejecución del solver
print(results)

# Salida formateada:
print("\nSolución:")
print(f"\tBeneficio = {instancia.Objetivo()}")

# OJO!!: Para el problema abstracto se trabaja con la instancia y se utiliza
    component_data_objects
# para acceder a las variables del problema.
print("\nVariables en el óptimo:")
for v in instancia.component_data_objects(pyo.Var, active=True):
    print(f"\t{v} = {v()}")

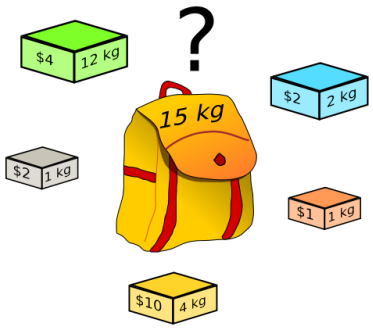

print("\nRestricciones en el óptimo:")
```

```python
for c in instancia.component_data_objects(pyo.Constraint, active=True):
    print(f"\t{c} = {c()}")

# Muestra la instancia sin resolver
#instancia.pprint()

# Muestra la instancia resuelta
#instancia.display()
```

A continuación resolveremos el problema concreto de la mochila con los datos del dibujo siguiente:



El fichero con los datos sería `Mochila_1.dat` (se puede descargar de la web de la asignatura):

```
## Ejemplo Mochila - Data en formato AMPL

set J:= Amarillo Naranja Azul Verde Gris;

param u:=
Amarillo 10
Naranja   1
Azul      2
Verde     4
Gris      2;

param p:=
Amarillo  4
Naranja   1
Azul      2
Verde    12
Gris      1;

param P:= 15;
```

Para obtener la solución se ejecutaría el siguiente código en el *prompt* de Anaconda:

```
python Mochila.py
```

4. Ejercicios

1. (**Gestión de Capital**) Una empresa dispone de 100 mil euros para invertir en 4 posibles proyectos: A, B, C y D. Cada uno de ellos tiene las siguientes características:

   Proyecto A: necesita una inversión de 40 mil euros y reportaría un beneficio de 10 mil euros,
   Proyecto B: necesita una inversión de 20 mil euros y reportaría un beneficio de 10 mil euros,
   Proyecto C: necesita una inversión de 14 mil euros y reportaría un beneficio de 8 mil euros,
   Proyecto D: necesita una inversión de 35 mil euros y reportaría un beneficio de 18 mil euros.

   a) ¿En que proyectos debería de invertir para que el beneficio sea máximo? Modeliza el problema y resuélvelo con la ayuda del pyomo.

b) En la solución óptima, ¿se invertiría en el proyecto que aporta más beneficio por euro invertido?

c) ¿Que restricción habría que añadir si para que se invierta en el proyecto D se tiene que invertir también en el proyecto C? Busca información de como se se podría añadir esta nueva restricción al modelo de Pyomo ya planteado y resuelve el problema.

2. Para el problema de Netflix que está en las transparencias del Tema 3, realizar las siguientes tareas:

   a) Crear un fichero en Pyomo con su modelo general asumiendo que las variables pueden tomar valores **reales** no negativos (tal y como está planteado en las transparencia).[1]

   b) Crear **otro** fichero en Pyomo con el mismo modelo general pero asumiendo que las variables solo pueden tomar valores **enteros** no negativos.

   c) Utilizando los dos ficheros de los apartados anteriores resolver el problema para los siguientes datos:

      - 3 servidores con capacidades de 3000, 2000 y 2500 MB, respectivamente,
      - 2 clientes con demandas de 3500 y 4000 MB, respectivamente,
      - Las tasas de transmisión (en milisegundos por MB) entre servidores y clientes vienen determinadas por la siguiente tabla:

	C_1	C_2
S_1	6	3
S_2	2	3
S_3	5	4

   d) ¿Hay diferencias entre las soluciones obtenidas utilizando el modelo con variables reales y el modelo con variables enteras?

   e) Resuelve 3 problemas más (con conjuntos de datos creados por vosotros) utilizando los dos modelos y comprobar si siguen dando las mismas soluciones entre ellos (los modelos).

3. Plantea la generalización del problema de la mochila donde existen s_j unidades de cada objeto j y hay que decidir cuantas unidades de cada uno se deben incluir.

   Resuelve el ejemplo de la Sección 3 en donde se podrían incluir hasta 2 unidades del objeto amarillo, 3 del naranja, 4 del azul, 1 del verde y 3 del gris. ¿Cuantas unidades de cada objeto se meterían en la mochila y cual sería el beneficio máximo?

[1] Este apartado ya estaba planteado como ejercicio en la práctica 2.